Curso GO Banorte Arquitectura TI

Instalación.

Preparando entorno

Ambiente de trabajo

Configuración.

Uno de los objetivos clave del diseño de Go es la adaptabilidad del código

Ambiente de trabajo

Patrones de concurrencia

1) Preparando instalación

1) Instalamos Git Bash, en git-scm.com

2) Instalamos normal mente Git Bash, e iniciamos para poder ver la terminal Git Bash.
3) Instalamos GO desde la página de Golang.

Download and install

1. Go download.
2. Go install.
3. Go code.

Download and install Go quickly with the steps described here.

For other content on installing, you might be interested in:

- Managing Go installations -- How to install multiple versions and uninstall.
- Installing Go from source -- How to check out the sources, build them on your own machine, and run them.

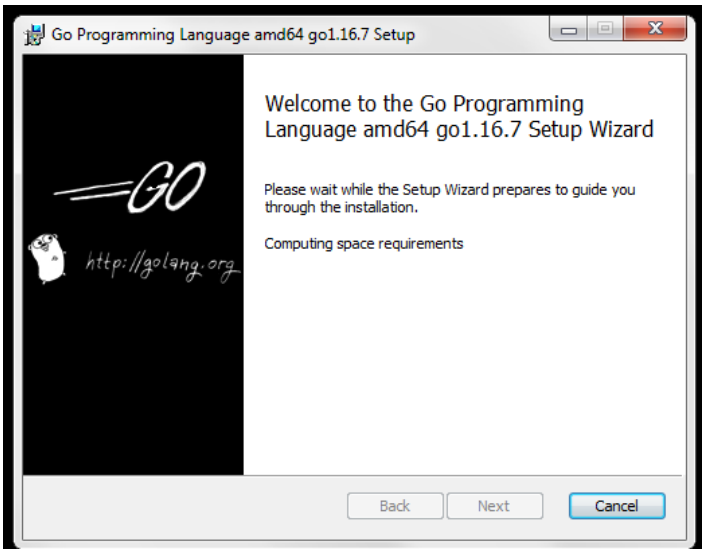
1. Go download.

Click the button below to download the Go installer.

Download Go for Windows
go1.16.7.windows-amd64.msi (119 MB)

Don't see your operating system here? Try one of the other downloads.

4)



5) Elegimos la carpeta donde colocaremos nuestros proyectos. C:\Go\ Preparamos el entorno de trabajo:

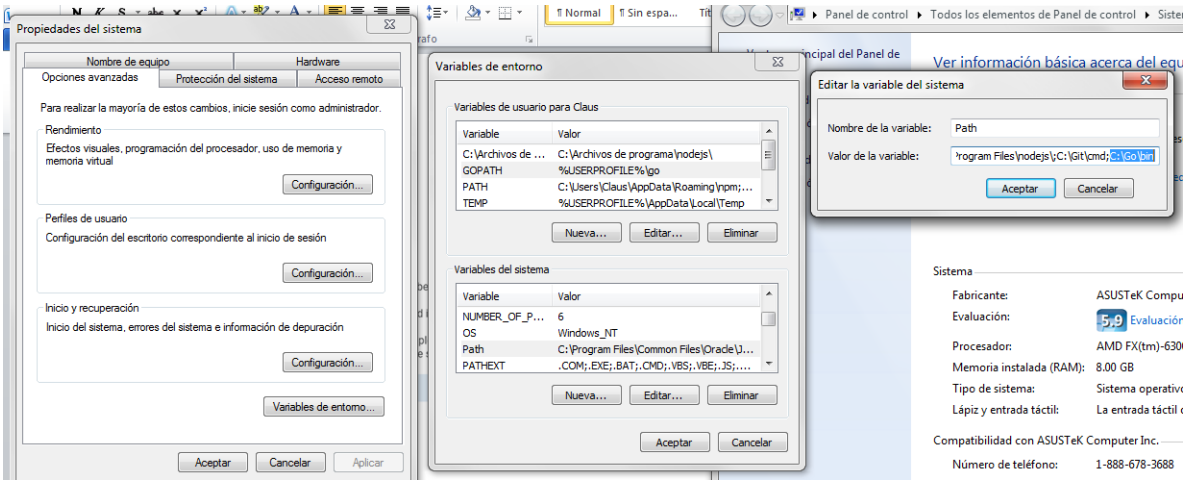
5.1 Ya que descargamos Git de Git-scm.com e instalamos, y le dimos a opciones referencia pro defecto nos muestra la terminal.

5.2 Una vez instalado editamos nuestras variables de entorno en:

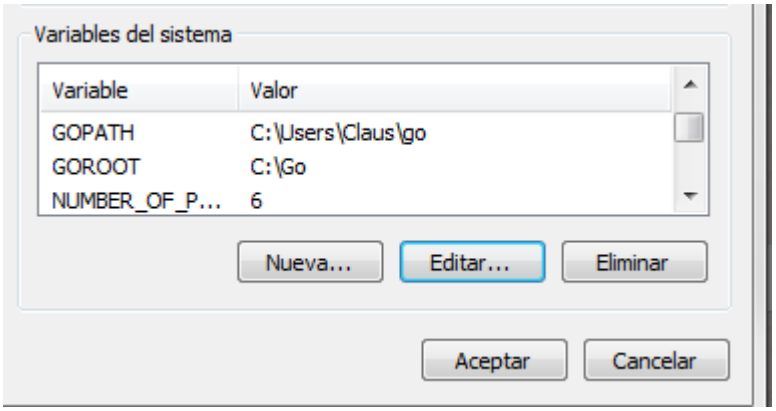
Mi PC -> Propiedades -> Configuración Avanzada del Sistema -> Variables de Entorno -> path -> editar -> nuevo

En variables del sistema -> path -> new.

Agregamos nuestra ruta, en mi caso es C:\Go\bin



Se ve así:



Igual mente para path se agrega la ruta, en mi caso es c:/Go/bin

2) Crear Entorno de trabajo

El código Go se debe mantener dentro de un ambiente de trabajo. Un ambiente de trabajo es una jerarquía de directorios con tres ramas en su raíz:

- src contiene los archivos de código fuente Go organizados en paquetes (un paquete por directorio)
- pkg contiene objetos paquete
- bin contiene las órdenes ejecutables.

En Git bash anotamos $ pwd para ubicarnos en que carpeta estamos

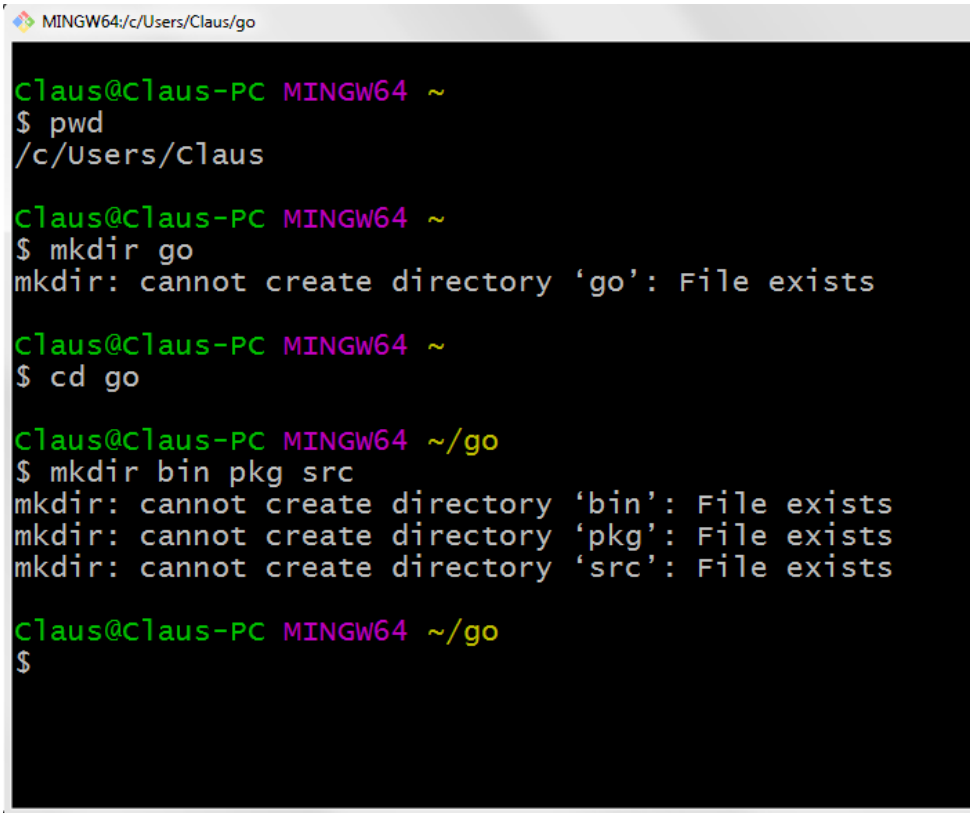
Creamos el directorio go (ya lo tengo ya creado en el ejemplo *)

Nos ubicamos en la carpeta /go y creamos 3 directorios

Bin: para códigos binarios

Pkg: para paquetes

Scr: aquí es donde trabajamos

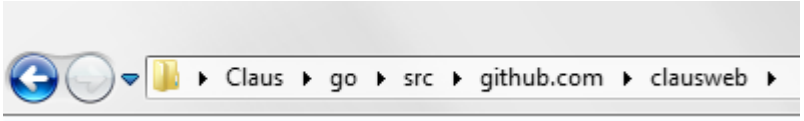


Ingresamos en src y creamos un directorio de nombre github.com (entiendo que no es necesario este nombre, pero es la recomendación del manual. Más adelante hablaremos del git y github)

Entramos en la carpeta github.com y creamos una carpeta con el nombre de usuario con el que estamos registrados en github.com De momento usaremos un ejemplo para cambiarlo después

```
$ cd src

claus@Claus-PC MINGW64 ~/go/src
$ mkdir github.com

claus@Claus-PC MINGW64 ~/go/src
$ cd github.com

claus@Claus-PC MINGW64 ~/go/src/github.com
$ mkd clausweb
bash: mkd: command not found

claus@Claus-PC MINGW64 ~/go/src/github.com
$ mkdir clausweb

claus@Claus-PC MINGW64 ~/go/src/github.com
$ cd clausweb

claus@Claus-PC MINGW64 ~/go/src/github.com/clausweb
$
```

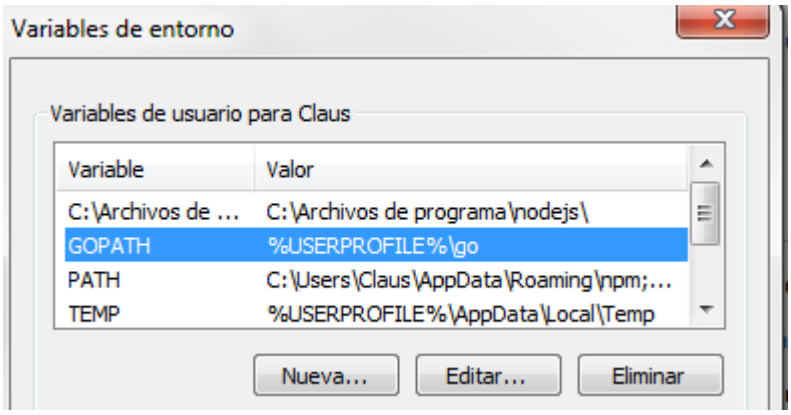
Creamos un primer proyecto: GoHackathon



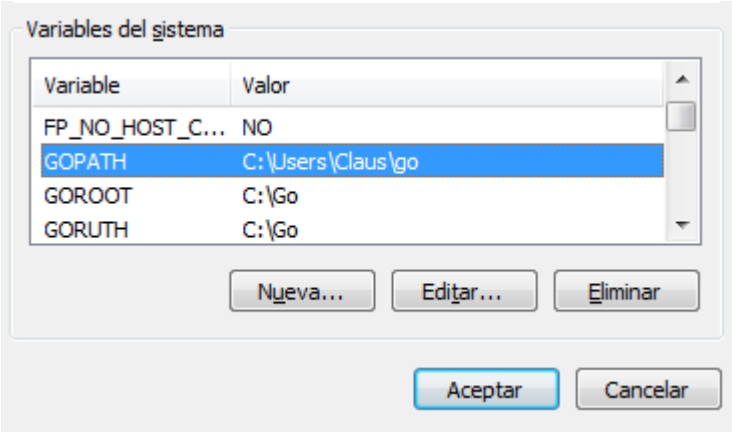
Ya echo este paso tenemos que crear nuestras variables de entorno

Vamos a my pc->propiedades->configuraciones avanzadas->variables de entorno

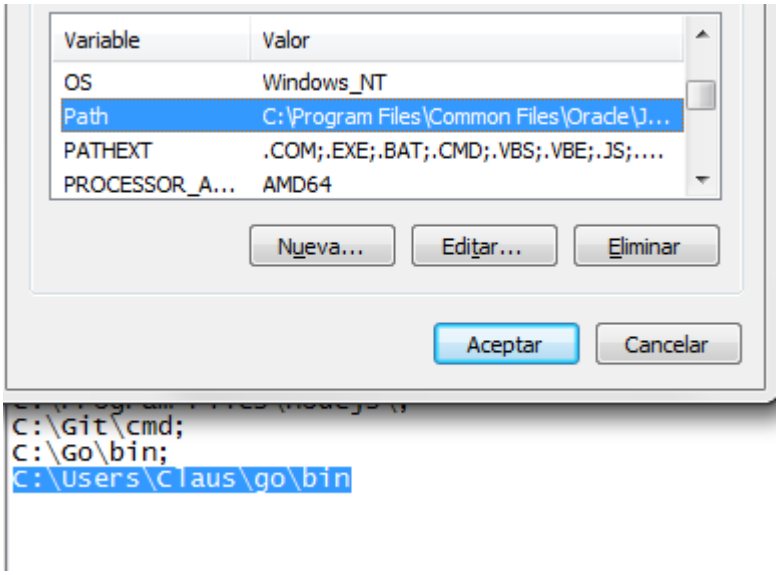
Aquí veremos que GOPATH tiene la carpeta de mi entorno de trabajo



Yo lo cree en mi variable de sistema:



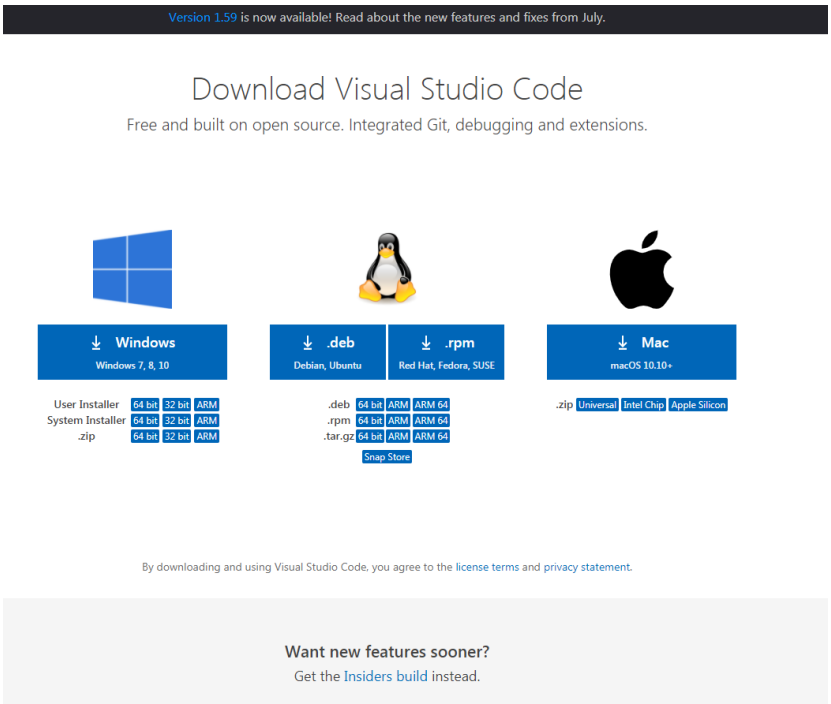
Luego, dentro de path agregamos la ruta bin:



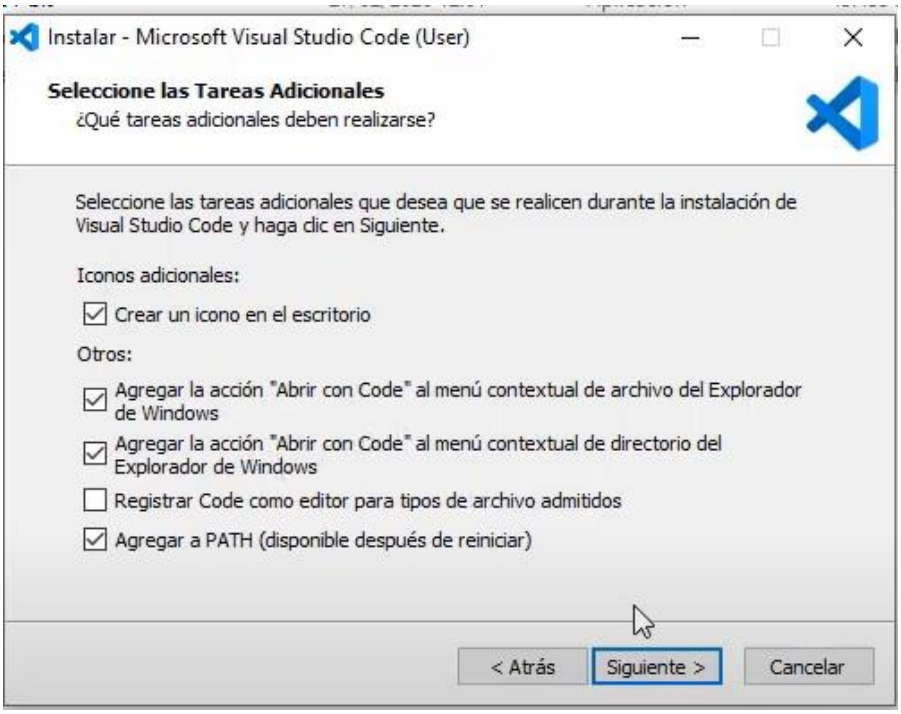
Con esto ya estaría terminado nuestro entorno de trabajo y podemos comenzar a trabajar

3 Editor de código Visual Estudio Code

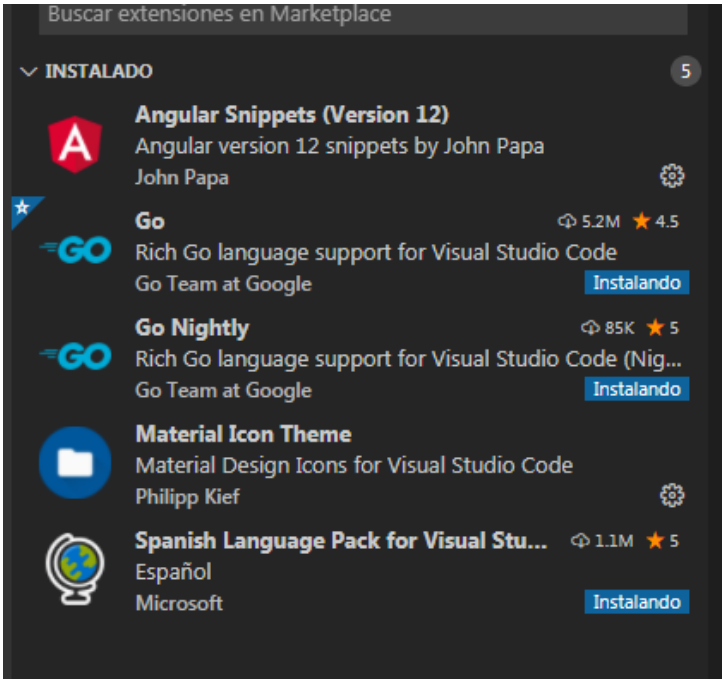
Es nuestro editor de código que encontramos en https://code.visualstudio.com/download



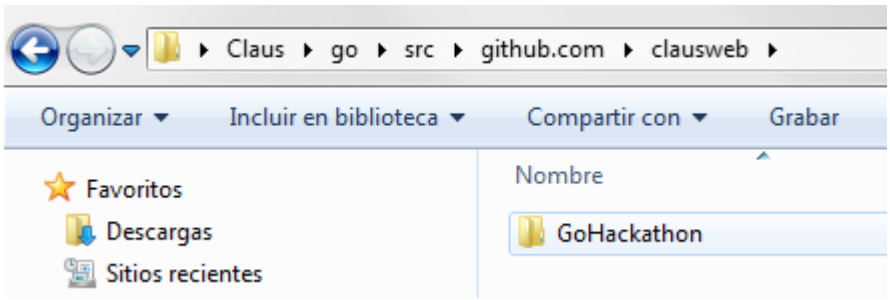
Instalamos normal mente, seleccionamos las opciones que nos permitan crear un icono de escritorio, abrir documentos con Visual Studio, y agregar path, y elegimos siguiente hasta finalizar y abrimos nuestro VS code.



En extensiones instalamos las opciones para poder trabajar con go:



Comprobamos con un proyecto abriendo con VS Code el archivo (ejemplo) C:\Users\Claus\go\src\github.com\clausweb



Para comprobar que todo funcione bien, la misma página de go nos muestra como comprobar la instalación con un ejemplo:

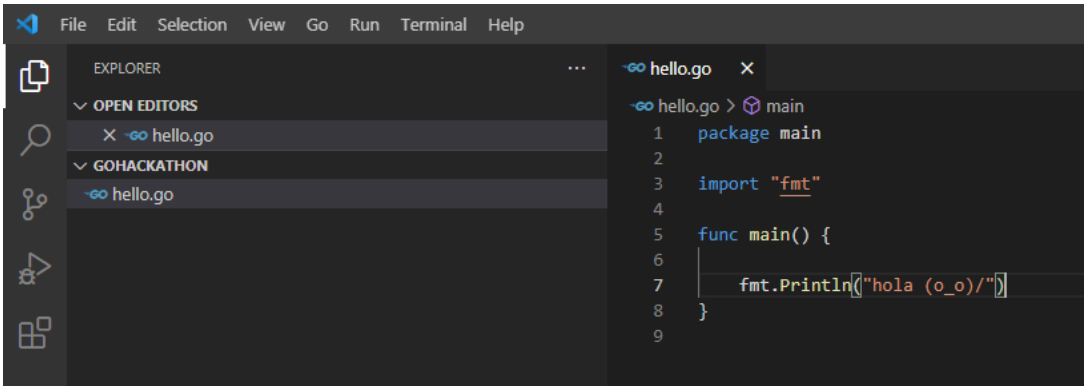
Test your installation

Check that Go is installed correctly by building a simple program, as follows.

Create a file named `hello.go` that looks like:

```
package main

import "fmt"

func main() {
	fmt.Printf("hello, world\n")
}
```

Copiamos en nuestro VS Code



En Git Bash nos colocamos en la carpeta correspondiente y agregamos “go run” al proyecto main.

NOTA: esto correrá el código pero no generara un código binario en la carpeta bin

```
claus@Claus-PC MINGW64 ~/go/src/github.com/clausweb
$ cd GoHackathon

claus@Claus-PC MINGW64 ~/go/src/github.com/clausweb/GoHackathon
$ go run hello.go
```

Si todo está bien nos muestra el texto que queremos

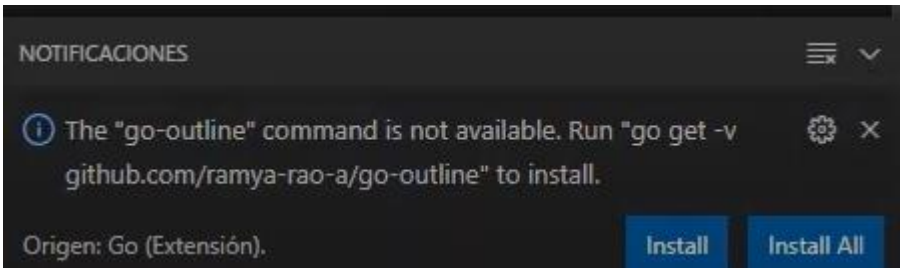
```
claus@Claus-PC MINGW64 ~/go/src/github.com/clausweb/GoHackathon
$ go run hello.go
hola (o_o)/
```

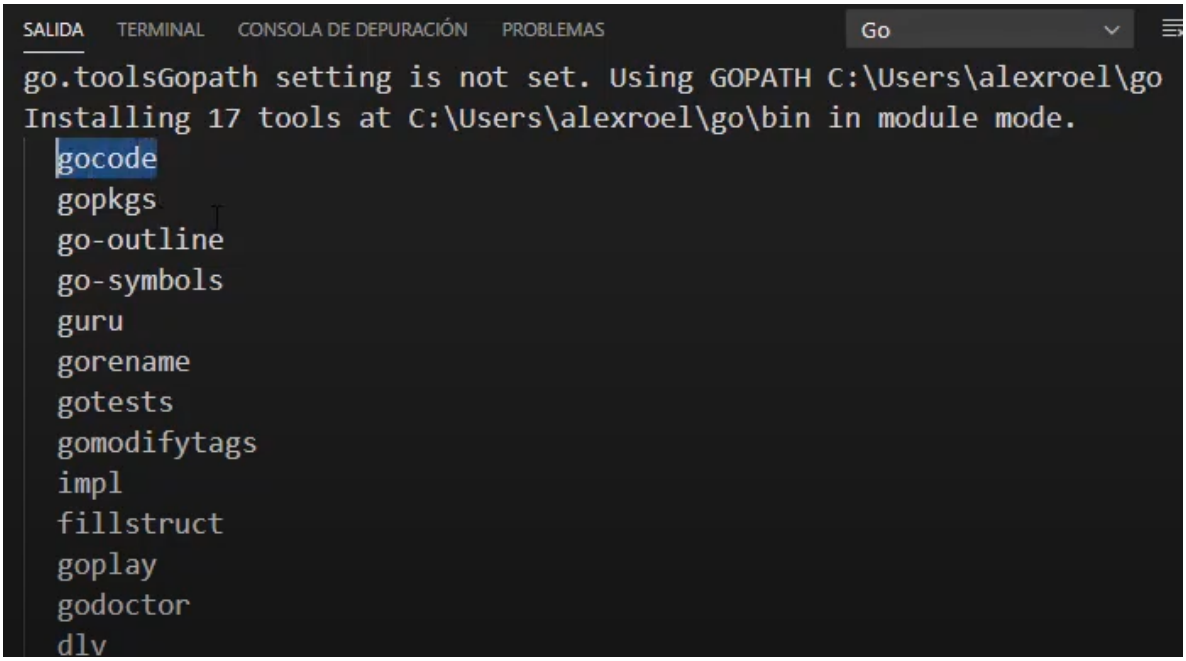
Cuando creamos un código binario con el comando build , no nos muestra resultado pero genera un .exe que corre el código

```
alexroel@DESKTOP-INTE6FS MINGW64 ~/go/src/github.com/alexroel/gocurso
$ go build hello.go

alexroel@DESKTOP-INTE6FS MINGW64 ~/go/src/github.com/alexroel/gocurso
$ ./hello.exe
hello, world
```

4 Herramientas Go

Al inicio VS code nos da la opción de instalar distintas herramientas





Esperamos a que se instale cada una y verificamos nuestro entorno de trabajo en las carpetas bin, pkg y src.

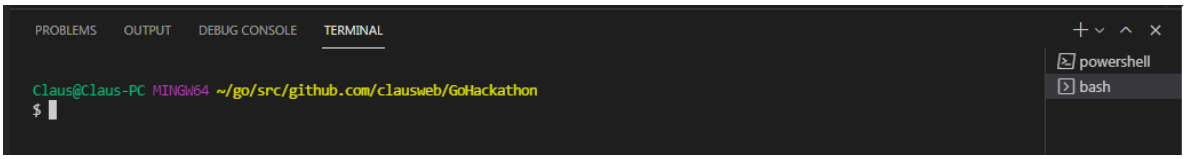
En bin veremos más contenido con nuestras herramientas de trabajo:

dlv	15/03/2020 14:58	Aplicación	17.653 KB
fillstruct	15/03/2020 14:56	Aplicación	6.857 KB
gocode	15/03/2020 14:54	Aplicación	12.756 KB
gocode-gomod	15/03/2020 14:59	Aplicación	12.311 KB
godef	15/03/2020 14:59	Aplicación	9.344 KB
godoctor	15/03/2020 14:57	Aplicación	8.081 KB
golint	15/03/2020 15:00	Aplicación	6.373 KB
gomodifytags	15/03/2020 14:56	Aplicación	4.769 KB
go-outline	15/03/2020 14:55	Aplicación	4.682 KB
gopkgs	15/03/2020 14:55	Aplicación	5.550 KB
goplay	15/03/2020 14:56	Aplicación	7.108 KB
gorename	15/03/2020 14:55	Aplicación	6.369 KB
goreturns	15/03/2020 15:00	Aplicación	7.990 KB
go-symbols	15/03/2020 14:55	Aplicación	4.481 KB
gotests	15/03/2020 14:55	Aplicación	12.686 KB
guru	15/03/2020 14:55	Aplicación	9.411 KB
impl	15/03/2020 14:56	Aplicación	6.071 KB

En pkg veremos los paquetes:

Nombre	Fecha de modificación	Tipo	Tamaño
mod	15/03/2020 14:59	Carpeta de archivos	
sumdb	15/03/2020 14:54	Carpeta de archivos	

Recordar que podemos seleccionar en nuestra consola de VS Code nuestro Git Bash o power Shell según necesitemos, es útil para ubicarse directamente



Para dar una idea del ambiente de trabajo en la practica este es un ejemplo:

```
bin
    hola                        # orden ejecutable
    yasalio                     # orden ejecutable
pkg
    linux_amd64
        github.com/gitnacho/ejemplo
            utilcadenas.a       # objecto paquete
src
    github.com/gitnacho/ejemplo
        .git/                   # metadatos del repositorio git
        hola
            hola.go             # fuente de la orden
        yasalio
            main.go             # fuente de la orden
            main_test.go        # fuente de la prueba
        utilcadenas
            reverso.go          # fuente del paquete
            reverso_test.go     # fuente de la prueba
```

Veamos un ejemplo practico de como ejecutar un programa go:

```go
package main

import "fmt"

func main() {
    fmt.Println("ya se go! /o,o/")
}
```

Ejecutamos usando la instrucción go run mas el nombre del programa

```
A2277852@CON-A2277852L MINGW64 ~/go/src/github.com/claudiovargas/calculorfc-go-master
$ go run main.go
ya se go! /o,o/

A2277852@CON-A2277852L MINGW64 ~/go/src/github.com/claudiovargas/calculorfc-go-master
$
```

Mediante este método podemos ver la ejecución del programa

Algunas veces nos hará falta construir el programa en vinario, para estos casos, usamos la instrucción "build", así después podremos correr el programa directamente en binario en el directorio "./main"

```
$ go run main.go
ya se go! /o,o/

A2277852@CON-A2277852L MINGW64 ~/go/src/github.com/claudiovargas/calculorfc-go-master
$ go build main.go

A2277852@CON-A2277852L MINGW64 ~/go/src/github.com/claudiovargas/calculorfc-go-master
$ ls
environment/  go.mod  main.exe*  main.go  README.md  servicios/  structs/  verificadores/

A2277852@CON-A2277852L MINGW64 ~/go/src/github.com/claudiovargas/calculorfc-go-master
$ ./main
ya se go! /o,o/
```

5 Paquetes

Iniciamos un programa creando en nuestro directorio:

$ mkdir $GOPATH/src/github.com/usuario/cursogo

Anotamos el siguiente código:

```go
package main

import "fmt"

func main() {
    fmt.Println("ya se go! /o,o/")
}
```

Comprobamos que el código compila con "build", esto **no** producirá un archivo.

Para ello usamos "go install que coloca el paquete dentro del directorio pkg del ambiente de trabajo

```
[Running] go run "c:\Users\A2277852\go\src\github.com\claudiovargas\cursogo\main.go"
ya se go! /o,o/

[Done] exited with code=0 in 1.106 seconds
```

Ahora puedes construir e instalar el programa con la herramienta go.

Puedes establecer la ruta: $ go install github.com/claudiovargas/cursogo

O Puedes colocarte directamente en el directorio y usar la instrucción: $ go install

```
A2277852@CON-A2277852L MINGW64 ~/go/src/github.com/claudiovargas/cursogo
$ go install

A2277852@CON-A2277852L MINGW64 ~/go/src/github.com/claudiovargas/cursogo
$
```

La herramienta go no genera ninguna salida mas que cuando se genera un error, si no nos regresa nada el programa ha corrido satisfactoriamente

CLAUDIO ALONSO VARGAS ORGANISTA › go › src › github.com › claudiovargas › cursogo

Nombre	Fecha de modificación	Tipo	Tamaño
main	17/08/2021 04:36 p. m.	Aplicación	2,047 KB
main	17/08/2021 04:17 p. m.	Archivo de origen ...	1 KB

Si estás utilizando un sistema de control de versiones, ahora sería un buen momento para iniciar un repositorio, añadir los archivos y enviar tu primera entrega. Una vez más, este paso es opcional: no es necesario utilizar el control de versiones para escribir código Go.

```
$ cd $GOPATH/src/github.com/claudiovargas/cursogo
$ git init
Inicia repositorio Git vacío en /home/user/go/src/github.com/claudiovargas/cursogo/main.git
$ git add main.go
$ git commit -m "entrega inicial"
[master (root-commit) 0b4507d] entrega inicial
 1 archivo cambió, 1 insercion(+)
 crea modo 100644 main.go
```

6 Bibliotecas

Iniciemos creando un directorio ejemplo:

$ mkdir $GOPATH/src/github.com/claudiovargas/utilcadenas

A continuación, crea un archivo llamado reverso.go en ese directorio con el siguiente contenido

```
//  utilcadenas este    paquete contiene    útiles  funciones   para    trabajar
//  con cadenas de  caracteres.
package utilcadenas

//  Reverso invierte    su  argumento   s   dejándolo   legible de  izquierda
//  a   derecha.
func Reverso(s string) string {
    r := []rune(s)
    for i, j := 0, len(r)-1; i < len(r)/2; i, j = i+1, j-1 {
        r[i], r[j] = r[j], r[i]
    }
    return string(r)
}
```

Ahora, comprueba que el paquete compila con go build :

$ go build github.com/claudiovargas /utilcadenas/reverso.go

Esto no producirá un archivo. Para ello, tienes que utilizar go install , que coloca el objeto paquete dentro del directorio pkg del ambiente de trabajo.

Después de confirmar la construcción del paquete utilcadenas , modifica tu programa main.go original (que está en $GOPATH/src/github.com/claudiovargas/cursogo) para utilizarlo:

```
package main

import (
    "fmt"

    "github.com/claudiovargas/utilcadenas"
)

func main() {
    fmt.Printf(utilcadenas.Reverso("!oG ,aloH¡"))
}
```

Siempre que la herramienta go instala un paquete o binario, también instala todas las dependencias que tenga. Por lo tanto cuando instalas el programa main.go

$ go install github.com/usuario/hola

Automáticamente también instalará el paquete utilcadenas . Al ejecutar la nueva versión del programa, deberías ver un nuevo mensaje, invertido:

```
¡Hola Go!
[Done] exited with code=0 in 2.676 seconds
```

Siempre que la herramienta go instala un paquete o binario, también instala todas las dependencias que tenga. Por lo tanto cuando instalas el programa

```
$ hola
¡Hola,  Go!
```

Después de los pasos anteriores, tu ambiente de trabajo se parecerá a este:

```
bin
                cursogo                         # orden ejecutable
pkg
                linux_amd64/                    # esto reflejará    tu  SO  y arquitectura
                github.com/claudiovargas
                            utilcadenas.a       #   objecto paquete
src
                github.com/claudiovargas
                            cursogo
                                main.go         #   fuente  de  la  orden
                utilcadenas
                            reverso.go          #   fuente  del paquet
```

Ten en cuenta que go install colocó el objeto utilcadenas.a en un directorio dentro de pkg/linux_amd64 que refleja tu directorio fuente. Esto es a modo de que futuras invocaciones a la herramienta go puedan encontrar el objeto paquete y evitar recompilar innecesariamente el paquete. La parte windows_amd64 está allí para ayudar en recopilaciones cruzadas, y refleja el sistema operativo y arquitectura de tu sistema. Las órdenes ejecutables se enlazan estáticamente; los objetos paquete no es necesario que estén presentes para ejecutar los programas Go.

Nombre de paquetes

La primera declaración en un archivo fuente de Go tiene que ser:

package nombre

Dónde nombre es el nombre predeterminado del paquete para importaciones. (Todos los archivos en un paquete tienen que utilizar el mismo nombre).

Una convención en Go es que el nombre del paquete es el último elemento de la ruta de importación:

el paquete importado como " crypto/rot13 " se debería llamar rot13 . Las órdenes ejecutables siempre tienen que usar package main . No hay ningún requisito para que los nombres de paquete sean únicos entre todos los paquetes enlazados a un solo binario, solo que las rutas de importación (sus nombres de archivo completos) sean únicos.

Probando

Go tiene una ligera plataforma de pruebas compuesta de la orden go test y el paquete testing. Escribes una prueba creando un archivo con un nombre que termine en _test.go el cual contiene las funciones llamadas TestXXX con la firma func (t *testing.T) . La plataforma de pruebas corre cada una de esas funciones; si la función invoca a una función de fallo tal como t.Error o t.Fail , se considerada que la prueba falló.

Añade una prueba al paquete utilcadenas creando el archivo

$GOPATH/src/github.com/claudiovargas/utilcadenas/reverso_test.go

conteniendo el siguiente código Go.

```
package utilcadenas
import "testing"
func    TestReverso(t   *testing.T) {
                casos   :=  []struct    {
                                ingresada,  deseada string
                }{
                                {"Hola, mundo", "odnum  ,aloH"},
                                {"Hola, O.o",   "o.O ,aloH"},
                                {"",    ""},
                }
                for _,  c   :=  range   casos   {
                                obtuve  :=  Reverso(c.ingresada)
                                if  obtuve  !=  c.deseada   {
                                                t.Errorf("Reverso(%q)   ==  %q, deseada %q",    c.ingresada,
obtuve, c.deseada)
                                }
                }
}
```

Entonces corre la prueba con go test :

```
$ go test github.com/claudiovargas/utilcadenas ok github.com/usuario/utilcadenas 0.165s
```

Como siempre, si estás corriendo la herramienta go desde el directorio del paquete, puedes omitir la ruta del paquete:

```
$ go test
ok github.com/claudiovargas/utilcadenas 0.165s
```

Paquetes remotos

Una ruta de importación puede describir cómo obtener el código fuente del paquete utilizando un sistema de control de versiones tal como Git o Mercurial. La herramienta go utiliza esta propiedad para automáticamente descargar paquetes desde repositorios remotos.

Por ejemplo, los fragmentos de código descritos en este documento también están mantenidos en un repositorio Git hospedado en Github, github.com/gitnacho/ejemplo. Si incluyes el URL del repositorio en la ruta de importación del paquete, go get lo descargará, construirá e instalará automáticamente:

```
$ go get github.com/gitclaus/ejemplo/hola
$ $GOPATH/bin/hola
¡Hola, ejemplos Go!
```

Si el paquete especificado no está presente en un ambiente de trabajo, go get lo colocará dentro del primer ambiente de trabajo especificado por GOPATH . (Si el paquete ya existe, go get omitirá la descarga remota y se comportará igual que go install).

Después de emitir la orden go get anterior, ahora el árbol de directorios del ambiente de trabajo se debería parecer a este:

```
bin
                hello                                                   #   orden   ejecutable
pkg
                linux_amd64
                                github.com/gitnacho/ejemplo
                                                utilcadenas.a           #   objeto  paquete
                                github.com/usuario
                                                utilcadenas.a           #   objecto paquete
src
                github.com/gitnacho/ejemplo
                                .git/                                   #   metadatos   del repositorio Git
                                hola
                                                hola.go                 #   fuente  de  la  orden
                                utilcadenas
                                                reverso.go              #   fuente  del paquete
                                                reverso_test.go         #   fuente  de  pruebas
                github.com/usuario
                                hola
                                                hola.go                 #   fuente  de  la  orden
                                utilcadenas
                                                reverso.go              #   fuente  del paquete
                                                reverso_test.go         #   fuente  de  pruebas
```

La orden hola hospedada en Github depende del paquete utilcadenas dentro del mismo repositorio. Las importaciones en el archivo hola.go utilizan la misma convención de las rutas de importación, por lo tanto la orden go get también es capaz de localizar e instalar el paquete dependiente

```
import "github.com/gitclaus/ejemplo/utilcadenas"
```

Esta convención es la manera más fácil de permitir que tus paquetes Go estén disponibles para que otros los utilicen. El Wiki de Go y godoc.org proporcionan listas de proyectos Go externos

Pánicos en tiempo de ejecución

Errores de ejecución como el intento de indexar un arreglo fuera de límites desencadena un pánico en tiempo de ejecución equivalente a llamar a la función incorporada panic con un valor de tipo interfaz runtime.Error definido en la implementación.

Ese tipo se ajusta al tipo interfaz error pre declarado. Los valores de error exactos que representan condiciones de error en distintos tiempos de ejecución no están especificados

```
package runtime
type    Error   interface   {
                error
                //  y   tal vez otros   métodos
}
```

Errores

El tipo predeclarado error se define como:

```
type error interface {
        Error() string
}
```

Es la interfaz convencional para representar una condición de error, el valor nil representa la ausencia de errores. Por ejemplo, una función para leer datos de un archivo se puede definir así:

```
func Lee(f *File, b []byte) (n int, err error)
```